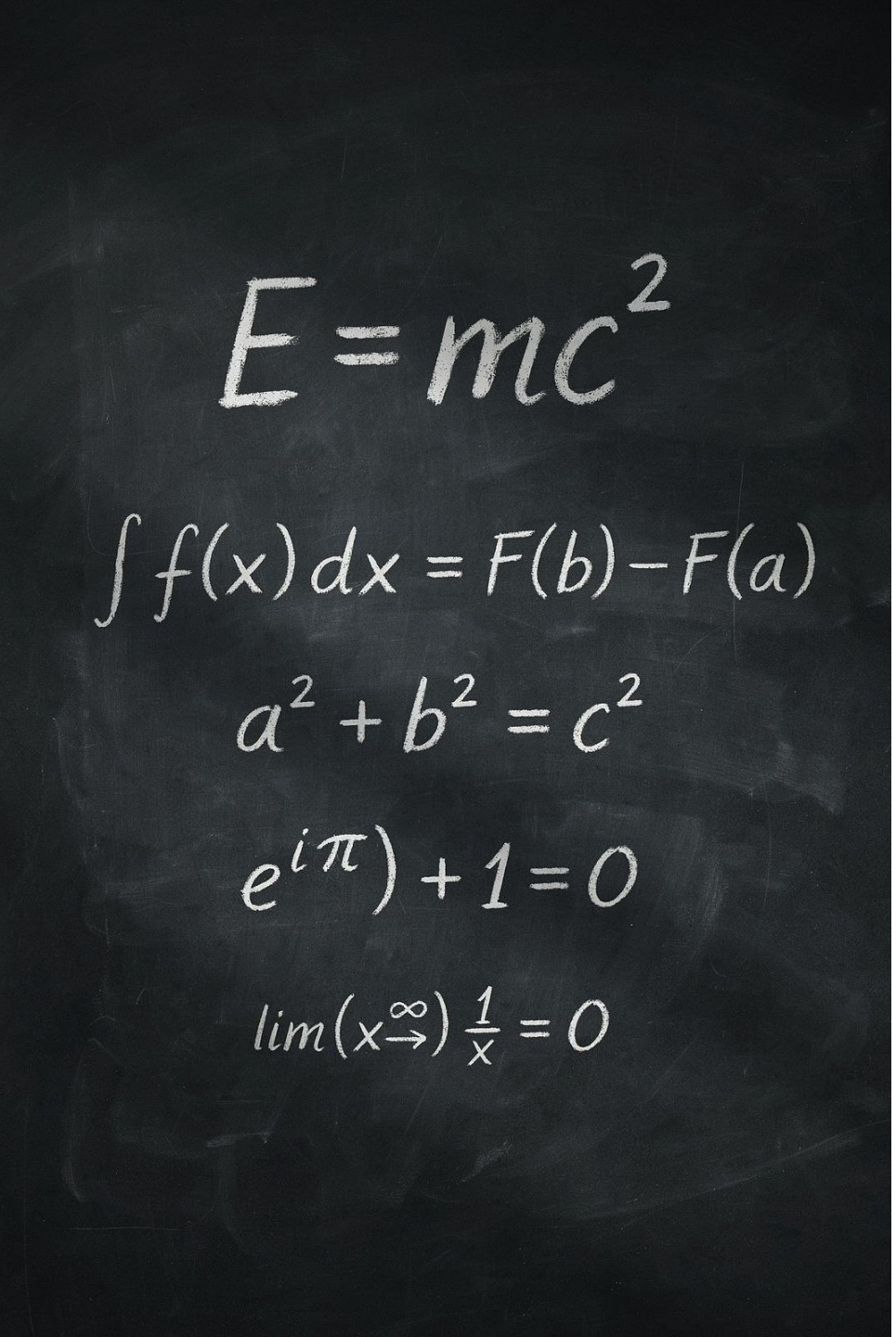


Matemáticas para IA

S2.1 · FUNDAMENTOS MATEMÁTICOS

Un recorrido esencial por el álgebra lineal, el cálculo y la optimización que hace posible el aprendizaje automático moderno. Desde vectores hasta descenso del gradiente, aquí están las herramientas que todo practicante de IA necesita dominar.

El lenguaje secreto de la IA



Datos como vectores

Una imagen, un texto o un sonido se representan como vectores numéricos. El álgebra lineal define cómo manipular estas representaciones de forma eficiente.



Aprendizaje como optimización

Entrenar un modelo es minimizar una función de error. El cálculo diferencial nos da la dirección de descenso hacia el mínimo óptimo.



Probabilidad y estadística

Los modelos aprenden distribuciones de datos. La probabilidad cuantifica la incertidumbre inherente en toda predicción.

Vectores: representación y notación

Notación matemática

Un vector en $\mathbb{R}^n$ se escribe como una columna de n componentes reales:

$$\vec{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}$$

Ejemplo en Python / NumPy

```
import numpy as np

# Vector en R^3
v = np.array([1.0, 2.5, -0.7])

# Norma (longitud)
norma = np.linalg.norm(v)
print(norma) # 2.70
```

¿Qué es una matriz?

Una **matriz** es una tabla bidimensional de números organizados en filas y columnas. Las matrices son la herramienta para representar transformaciones lineales, conjuntos de datos (donde cada fila es una muestra) o los pesos de una red neuronal.

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \in \mathbb{R}^{2\times 3}$$

Filas

Muestras de datos o ecuaciones del sistema

Columnas

Características o variables del problema

Dimensión

Se escribe como **m × n** (filas × columnas)

Producto escalar (dot product)

Definición

El producto escalar de dos vectores **u** y **v** de la misma dimensión es la suma de los productos de sus componentes:

$$\vec{u} \cdot \vec{v} = \sum_{i=1}^{n} u_i v_i = |\vec{u}||\vec{v}| \cos\theta$$

El resultado es un **escalar** (un único número). Mide cuánto se «alinean» dos vectores.

En IA, sirve para...

- Calcular similitud entre vectores de características
- Aplicar pesos en una neurona artificial: $z = \vec{w} \cdot \vec{x} + b$
- Medir la similitud del coseno en búsqueda semántica

Producto matricial

La operación más central del deep learning. Para multiplicar **A (m×k)** por **B (k×n)**, cada elemento de la matriz resultante **C (m×n)** es el producto escalar de una fila de A con una columna de B:

$$C_{ij} = \sum_{r=1}^{k} A_{ir} \cdot B_{rj}$$

Regla de dimensiones: el número de columnas de A debe coincidir con el número de filas de B. El resultado tiene las filas de A y las columnas de B.

Una capa densa de una red neuronal aplica exactamente: $\mathbf{y} = A\mathbf{x} + \mathbf{b}$, donde A es la matriz de pesos y x el vector de entrada.

Transpuesta e inversa

Transpuesta A^T

Intercambia filas y columnas. Si A es m×n, entonces A^T es n×m. Propiedad clave: $(AB)^T = B^T A^T$. Se usa constantemente en backpropagation.

Inversa A^{-1}

Solo existe para matrices cuadradas no singulares. Cumple $AA^{-1} = I$. Permite «deshacer» la transformación lineal. En la práctica, se evita calcularla directamente por coste computacional.

Pseudoinversa A^+

Generalización de la inversa para matrices no cuadradas. La solución de mínimos cuadrados es $\hat{x} = A^+ b$, fundamental en regresión lineal.

La idea intuitiva

Definición

Sea A una matriz cuadrada. Un vector **v ≠ 0** es un **autovector** de A si al multiplicarlo solo cambia de escala, no de dirección:

$$A\vec{v} = \lambda\vec{v}$$

El escalar λ (lambda) se llama **autovalor** asociado a **v**.

Aplicaciones en IA

- **PCA**: los autovectores de la matriz de covarianza definen las direcciones de máxima varianza
- **PageRank**: la página web más relevante es el autovector dominante del grafo de la web
- **Redes recurrentes**: los autovalores determinan la estabilidad del entrenamiento

La derivada: tasa de cambio instantáneo

Definición formal

La derivada de f en el punto x mide cuánto cambia f ante una variación infinitesimal de x:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Geométricamente, es la **pendiente de la recta tangente** a la curva en ese punto.

Derivadas comunes en IA

Función	Derivada
x^n	nx^{n-1}
e^x	e^x
$\ln(x)$	$1/x$
$\sigma(x)$	$\sigma(x)(1-\sigma(x))$

Derivada parcial: un paso a la vez

La **derivada parcial** de $f(x_1, x_2, \ldots, x_n)$ respecto a x_i mide cómo cambia f cuando solo varía x_i, manteniendo las demás constantes. Se denota $\frac{\partial f}{\partial x_i}$.

Ejemplo

$$f(x, y) = 3x^2 + 2xy + y^3$$

$$\frac{\partial f}{\partial x} = 6x + 2y$$

$$\frac{\partial f}{\partial y} = 2x + 3y^2$$

Intuición

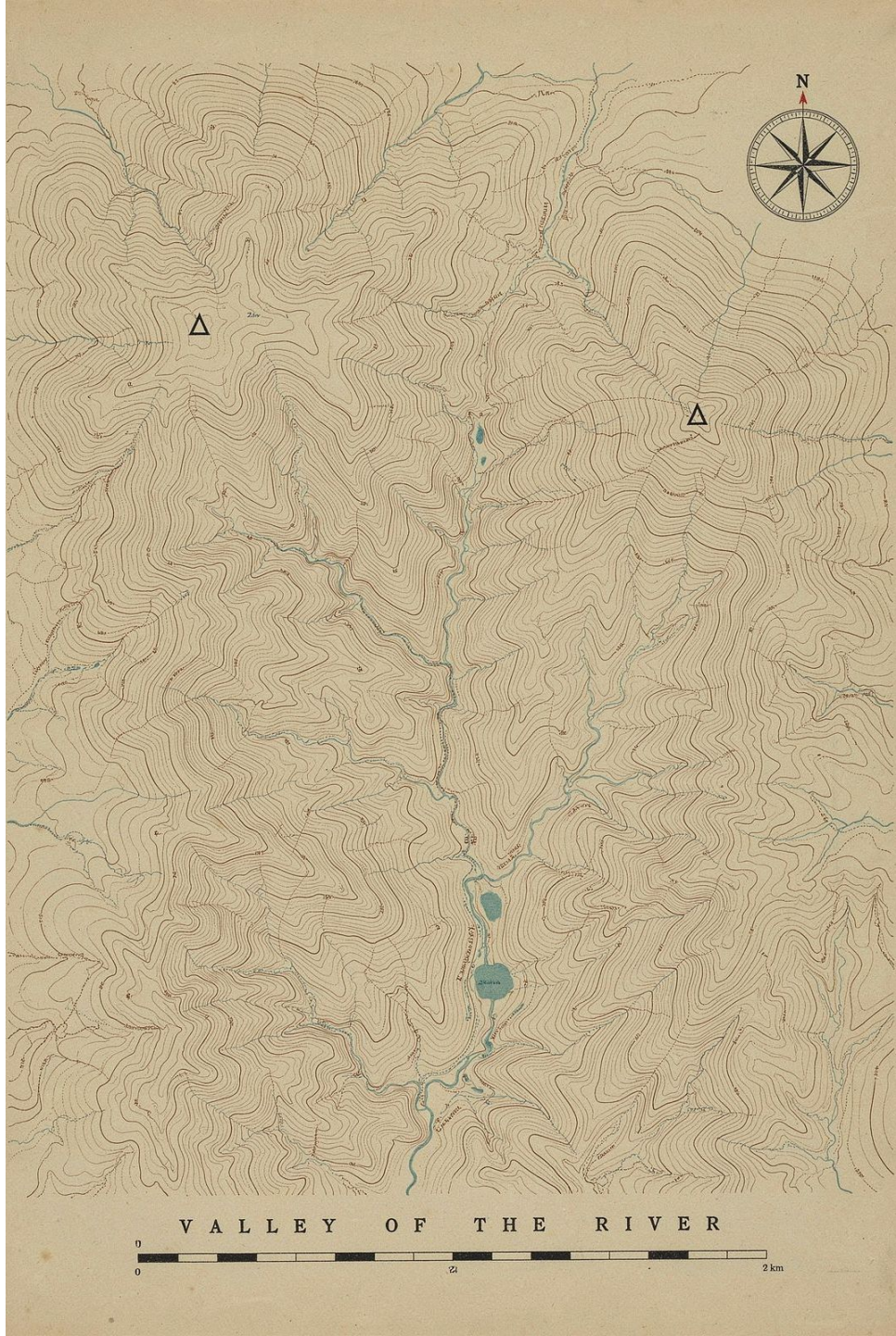
Imagina que estás en una montaña. La derivada parcial respecto a x te dice cuánto subes o bajas si te mueves únicamente hacia el Este, ignorando el Norte. La derivada respecto a y hace lo mismo pero en dirección Norte.

Derivadas parciales y gradiente

El gradiente: la brújula del aprendizaje

El **gradiente** agrupa todas las derivadas parciales en un solo vector. Apunta siempre en la dirección de **máximo crecimiento** de la función. En optimización, nos moveremos en la dirección *opuesta* para minimizar el error:

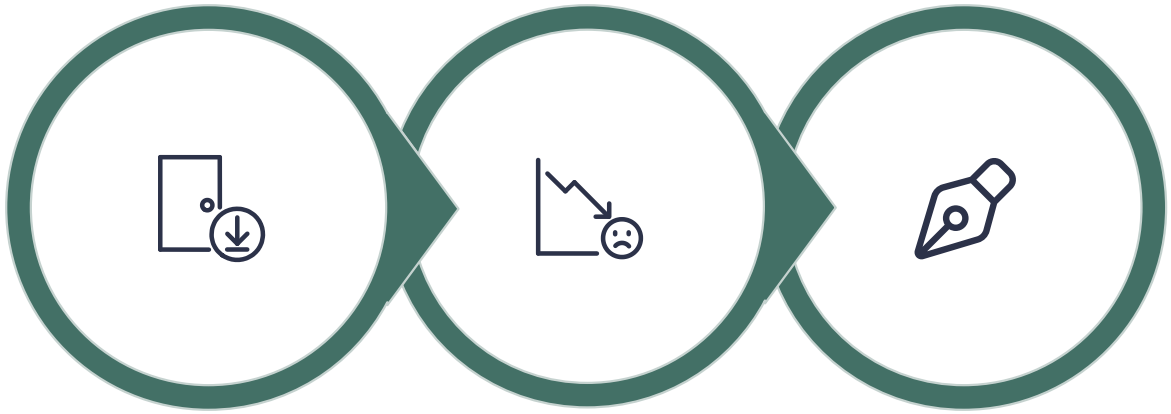
$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right)$$



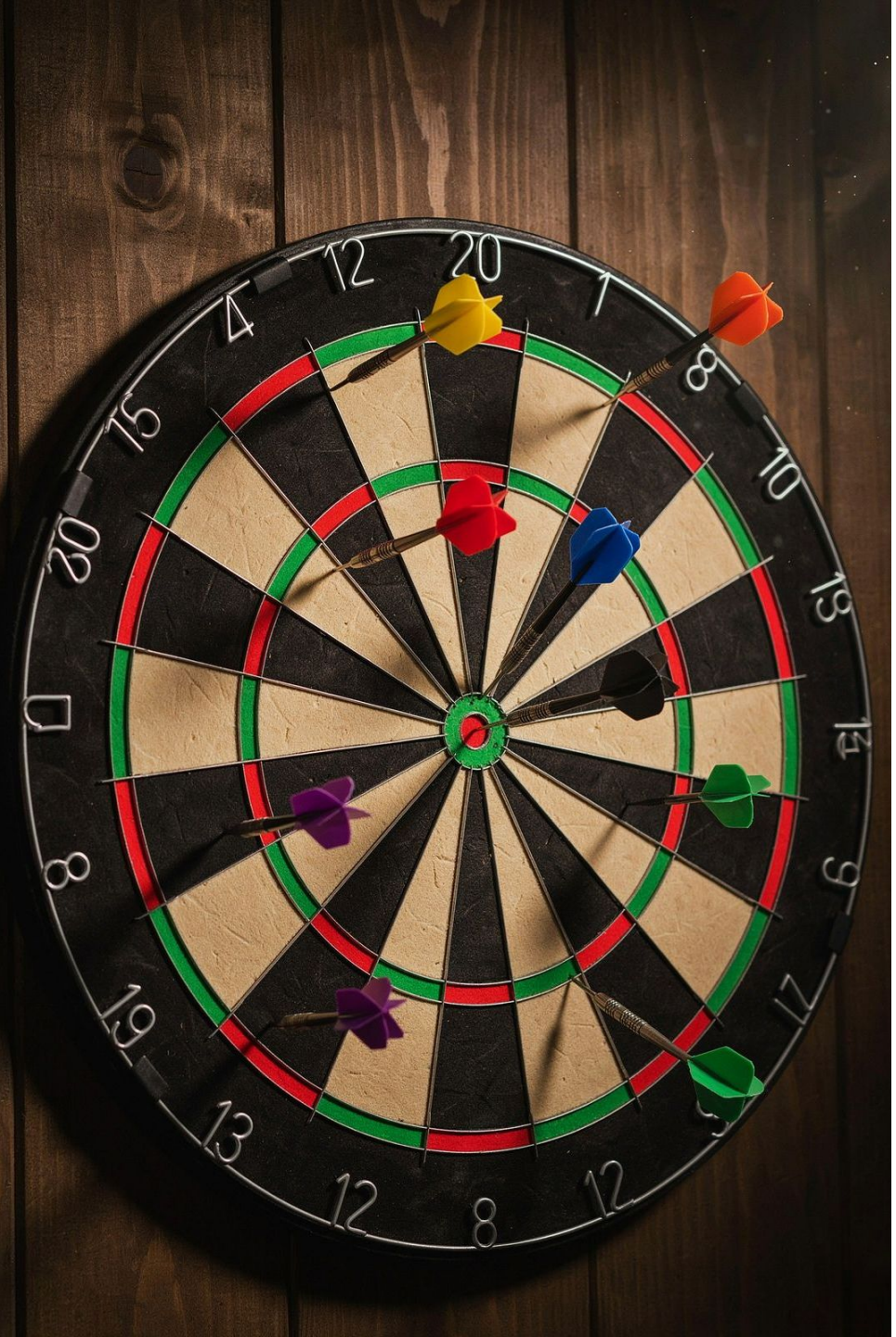
Regla de la cadena: composición de funciones

Cuando una función se compone de varias funciones anidadas — exactamente como ocurre en las capas de una red neuronal — la regla de la cadena nos permite derivar el todo a partir de las partes:

$$\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$$



Entrada Salida Retroprop.



¿Qué es la función de coste?

La **función de coste** (también llamada función de pérdida o *loss function*) cuantifica numéricamente la diferencia entre las predicciones del modelo y los valores reales. Cuanto mayor es el coste, peor es el modelo. Entrenar consiste en encontrar los parámetros que **minimizan** esta función.

Cómo funciona el descenso del gradiente

La regla de actualización

En cada iteración, los parámetros se mueven una pequeña cantidad en la dirección opuesta al gradiente:

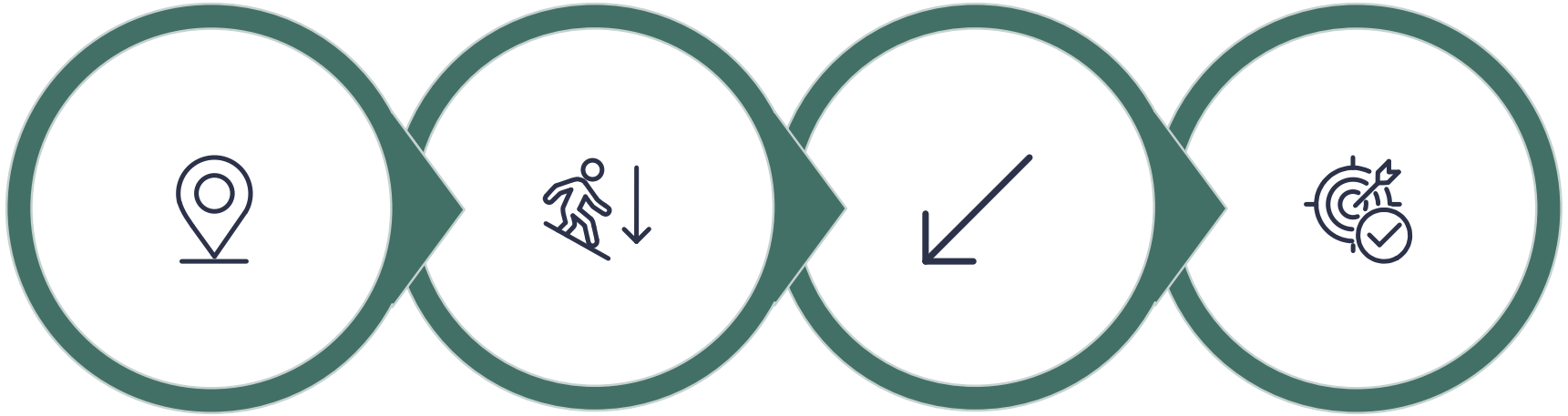
$$\theta \leftarrow \theta - \alpha \cdot \nabla_\theta L(\theta)$$

Donde α es la **tasa de aprendizaje** (*learning rate*), un hiperparámetro crítico.

Paso a paso

1. Inicializar los pesos θ aleatoriamente
2. Calcular el coste $L(\theta)$ con los datos actuales
3. Calcular el gradiente $\nabla_\theta L$ via backprop
4. Actualizar θ con la regla anterior
5. Repetir hasta convergencia

Visualizando el descenso del gradiente



Punto aleatorio

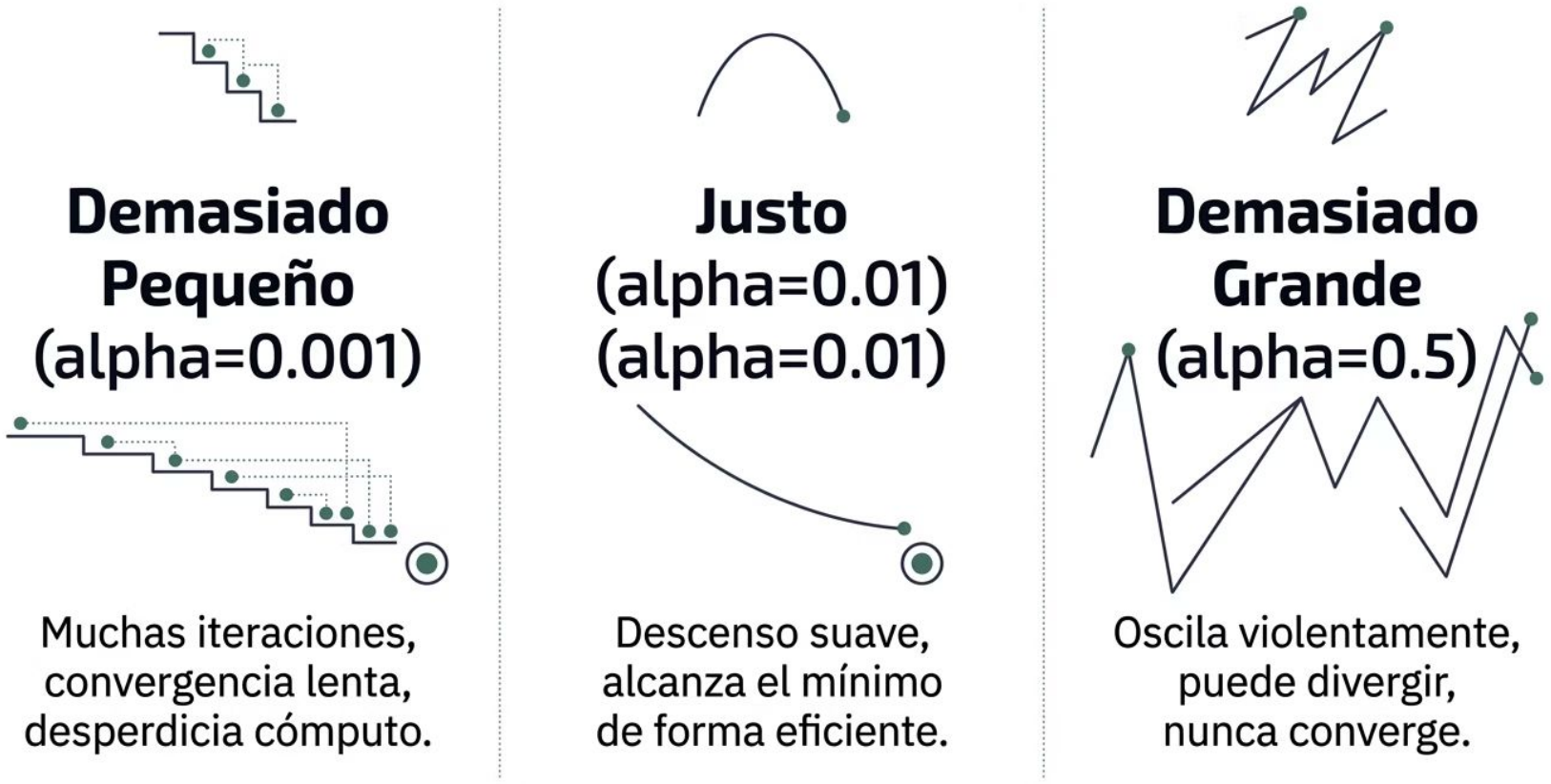
Calcular gradiente

Dar paso

Converger

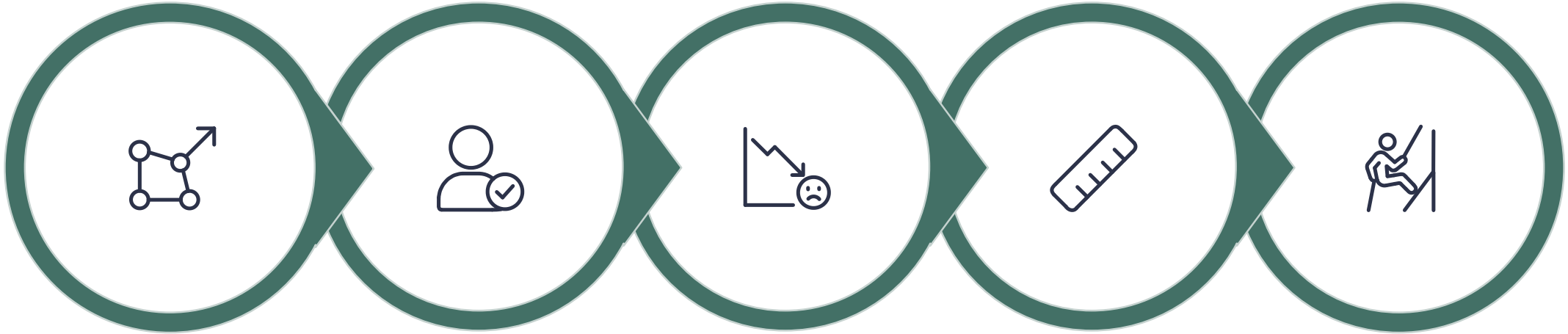
La tasa de aprendizaje α controla el tamaño del paso: demasiado pequeña y la convergencia es lenta; demasiado grande y el algoritmo puede oscilar o divergir, saltando por encima del mínimo.

El efecto de la tasa de aprendizaje



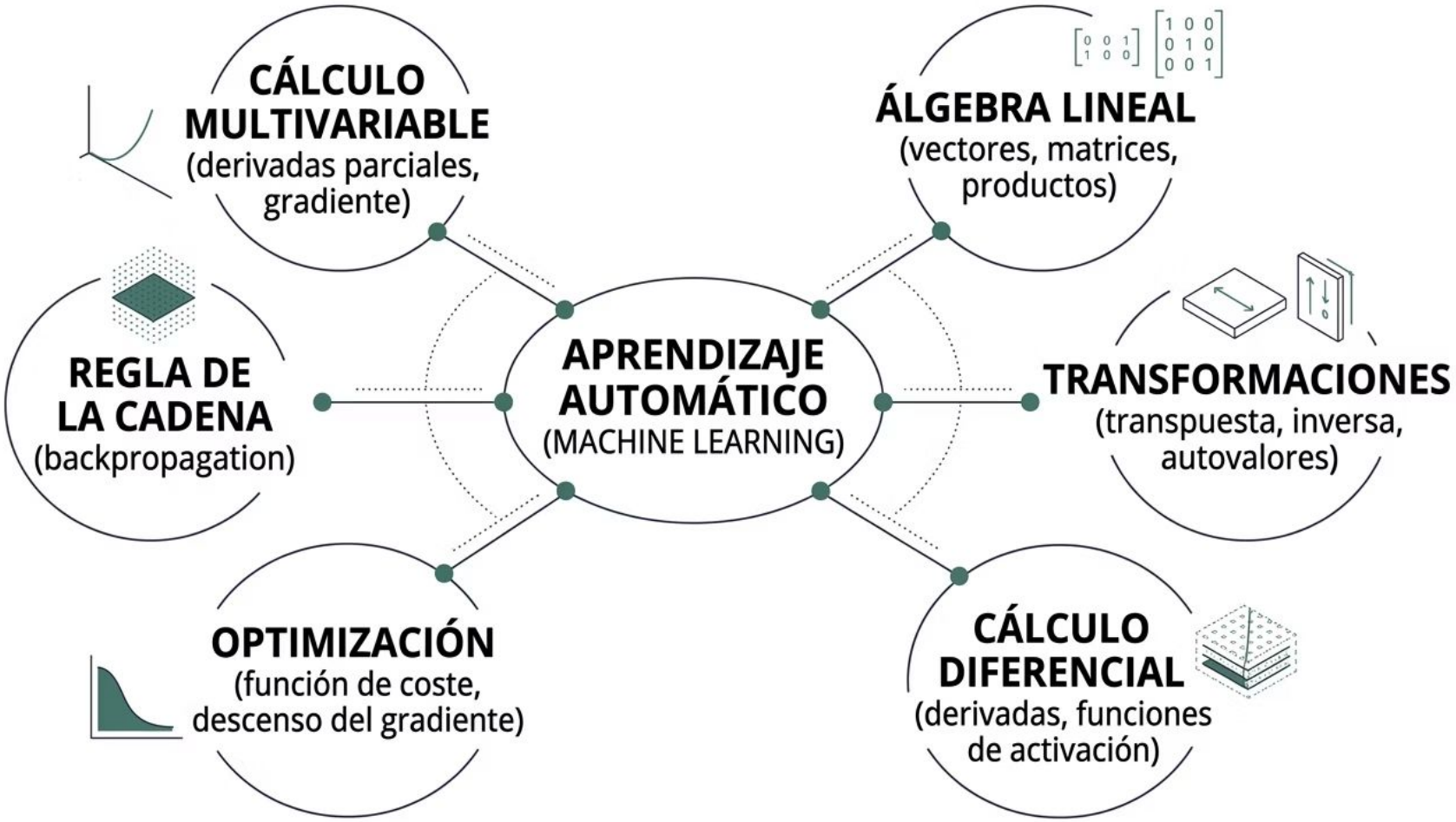
Encontrar la tasa de aprendizaje adecuada es uno de los ajustes más importantes en el entrenamiento. Técnicas como el **learning rate scheduler** y el **warm-up** permiten variarlo dinámicamente durante el entrenamiento.

Conectando todo: de los datos al modelo



Datos Propagación Pérdida Retroprop. Actualiza

Resumen: el mapa matemático de la IA



Conceptos clave de la sesión

Vectores y matrices

Representación universal de datos y transformaciones en IA

Gradiente

El vector que señala la dirección de máximo crecimiento (o caída) de cualquier función

Regla de la cadena

Fundamento matemático del algoritmo de backpropagation en redes profundas

Descenso del gradiente

Algoritmo iterativo que minimiza la función de coste ajustando los pesos del modelo